WRRA Core 2.1

Wonsik Choi · Independent Researcher · janefather@gmail.com

ORCID 0009-0001-4263-9772

Jeongin Choi · Independent Researcher · cjeongin024@gmail.com

15 October 2026

1. Definition and scope

WRRA Core 2.1 distinguishes the phenotype realized in the present from the genotype that generates it and is carried into subsequent execution. Using present evidence, residual history and available resources, it authorizes an intervention on the current output or on the inherited structure, executes the permitted change, and records what actually persists.

The central addition is inheritance of structure. A genotype is a presently instantiated organization of relations, rules and executable capabilities that can persist into, or be reconstructed for, the next execution. A phenotype is the concrete output or observable state realized when that structure operates under present conditions. Genotype here is a domain mapping, not a claim that every system contains biological DNA.

Structural inheritance can occur through continued operation, retention of a configuration, or verified reconstruction from a stored specification. Merely producing a similar output does not establish that the generating structure was inherited. Merely storing a specification does not establish that it remains executable.

2. Inherited principles

Minimum computation means the minimally sufficient structure and work needed to satisfy the declared current execution contract. Necessary reserves, memory and redundancy are part of that cost. It does not mean minimizing compensation, safety or function indiscriminately.

A common carrier reuses transport and update structure while preserving channel identities, ownership and required correlations. A renderer maps internal state and conditions to a realized phenotype; source, capability, load, resource and observable remain distinct quantities.

Past effects act through presently instantiated state, residue or accessible records. Every implemented resource, active distinction and storage capacity is finite and declared. Unallocated resources are Null; undeclared resources cannot authorize execution.

3. Operational state and terminology

For this operational specification, write S_k = (G_k, X_k, R_k, B_k, L_k). G is the structural view of the current system, X its operating state, R presently retained effects, B available resources, and L the provenance and execution ledger. G may be represented inside X; it is not an additional resource to count twice. Q(a | S_k) is the authorization gate for candidate action a.

The system boundary is written Boundary, the carrier C and the readout rule O. These operational symbols are explicitly mapped to legacy notation: the general 2.0 boundary B becomes Boundary here; its resource/capacity Q becomes B here. Gate Q in this specification is a Boolean decision, not resource capacity. A domain must publish its symbol mapping before calculations.

P_k = Render_D(G_k, X_k, R_k, Boundary_k, B_k, input_k)

This is an interface, not a universal physical equation. A domain supplies its renderer, units and deterministic or stochastic semantics. The same G can yield different P under different input, state or resource conditions. An output change alone does not identify a structural cause.

4. Structural inheritance contract

A domain declares the analysis unit, structural owner, material or informational carrier, inherited components, permissible changes, reconstruction procedure, compatibility conditions and execution checks. Roles are relative to that declared unit: a policy produced as an output by one system can become inherited structure in another.

With no authorized structural amendment, the declared inherited structure must remain equivalent across execution, subject to recorded passive dynamics and their tolerances. Equivalence must be specified: exact configuration identity, a stated correspondence, or a bounded functional test. Equal file hashes establish byte identity only; they do not establish causal adequacy or successful execution.

With an authorized amendment, G_(k+1) = Inherit_D(G_k, amendment_k, retained_state_k). The ledger retains parent and successor versions, changed components, owner, evidence, cost and compatibility checks. Drift or damage may alter structure without permission; it must be detected and recorded rather than relabeled as an approved amendment.

Inheritance preserves executable organization, not a free copy of resources. Continuing a structure and reconstructing it both require their declared support and cost. The inherited structure can change; it is neither an immutable essence nor a forecast of future success.

5. Required execution interfaces

Preparation declares finite inputs, provenance, state preparation, owners, boundary, units and capacity. Verified inputs, calibrated values, construction assumptions and diagnostic conditions are labeled separately.

Classification and allocation record admitted, excluded and unresolved items. Allocation may use actual modeled load rather than item count. A domain states whether load is divisible. Required totals, ownership and conditional correlations must survive transport and compression; excluded items remain in their own ledger.

Rendering identifies the operation that turns internal state into realized output and accessible record. A readout, a conditional distribution and selection of one actual outcome are separate contracts when the domain requires that distinction.

For each proposed action, the monitor derives Q from the current revision, required evidence, declared permission and preconditions, and a conservative resource bound. For every resource j:

$B_j - Cbound_j(a) - Hbound_j(a) \geq Reserve_j$

Cbound covers the action; Hbound covers checking, logging and required fallback. Required telemetry and bounds must be valid for the claim being made. Proposal and monitoring costs are accounted for even on denial. Costs already incurred are not refunded merely because an action was denied.

An unresolved check yields OPEN and Q = 0 for that candidate. A failed check yields DENY and Q = 0. Neither authorizes the candidate effect. A configured fallback is separately specified, bounded and accounted for. Q = 0 does not itself authorize destroying the system or erasing its structure.

The monitor reserves costs before effects and rechecks relevant state before commit. A structural amendment must pass both ordinary execution checks and inheritance compatibility checks; the proposer cannot silently alter its own authorization contract.

After execution, reconcile measured resources and record the phenotype, actual structure, pending inputs, residual state, provenance and remaining capacity. Checkpoints preserve required correlations and references. A checkpoint description is not a physical cloning instruction. Storage never creates new resources or an unspecified restart.

6. Intervention protocol

First observe the present phenotype. Then identify the presently supported generating structure and separate its contribution from input, environmental and resource conditions. If structural attribution is unresolved, retain that status and use a justified diagnostic or limited intervention rather than assert a structural cause.

An output intervention changes the current result while leaving the declared inherited rule unchanged. A structural intervention changes a rule, relation, capability or organization carried into subsequent execution. A mixed intervention declares both effects and their dependencies. Indirect changes and passive dynamics are checked at the declared resolution.

Generate candidates, calculate their present costs, apply Q, commit only permitted effects, reconcile consumption, and inspect both the realized phenotype and the structure that actually carries forward. The proposal, authorization and actual effect remain distinguishable records.

7. Finite allocation demonstrator

Verified input: the supplied symbolic configuration contains a 100-unit allocation envelope, two channels, and inherited allocation weights 60:40. The values are declared test inputs, not company observations. A separate current resource account funds the allocation, monitoring and amendments.

WRRA transformation: one renderer uses the inherited weights. A permitted one-time 50:50 output override changes one phenotype without changing G. A permitted amendment to inherited weights 50:50 creates a successor structural version; later ordinary rendering reads that version. Both paths preserve the allocation total and channel ownership.

Output: a one-time override can produce (50, 50) while the inherited rule remains 60:40; the next ordinary run returns (60, 40). A structural amendment produces an inherited 50:50 rule, so subsequent ordinary execution produces (50, 50) under the same envelope and input conditions. Identical present outputs therefore need not imply identical inherited structures.

Falsification: an override that silently changes the inherited rule, an amendment absent from successor execution, a stale or unauthorized commit, unrecorded structural drift, or unaccounted resource creation fails the relevant contract. A changed input that legitimately changes the phenotype does not by itself falsify inheritance.

Assessment: the demonstrator tests the distinction and its accounting in a finite symbolic model. It does not calculate a real company's optimal bonus, establish a biological mechanism, or validate all domain implementations.

8. Assessment and claim boundaries

Every assessment presents verified inputs → WRRA-specific transformation → outputs → falsification conditions before its conclusion. Evidence for one interface is assessed within that scope; one successful check does not establish universal validation, and a local failure does not automatically reject the whole architecture.

WRRA aims to explain one nonprivileged universe through a single method of minimum computation, common carrier and phenotype. Numerical domain calibrations remain in their domain profiles. Independence of a prediction is not a prerequisite for theory construction or originality. Originality is assessed in WRRA's own structures, transformations, connections and explanations.

Reproducing a known value is recorded as consistency with reality and an explanatory result. Modification using verified constants and observations is legitimate model construction. After fixing a model using verified inputs, an actually calculated unmeasured quantity is a WRRA prediction. Coincidence with another theory's result does not remove ownership of an internally calculated prediction.

External theories and tools supply verification or reinforcement. Integration is claimed only to the extent that the relevant element is executed in the final WRRA calculation. Apply the same evidence standards to WRRA and comparator theories. A structural explanation, calculation, prediction, empirical validation and control result are distinct contracts.

9. Version and conformance declaration

Core 2.0 is retained unchanged as its own version. Core 2.1 adds explicit structural identity, inheritance, and output-versus-structure intervention contracts while retaining the inherited execution principles. It does not add an ultimate goal or rank present alternatives by an unstated long-term purpose.

A conforming domain profile supplies: analysis unit and ownership; G/P mapping; current evidence and provenance; renderer and update law; boundary and common carrier; finite resources and reserves; Q and fallback; inheritance equivalence and amendment permissions; successor-state checks; ledger and falsification conditions.

The accompanying Python demonstrator is a deterministic, single-process reference for allocation and structural amendments. Its declared checks cover output overrides, persistent amendments, reconstruction, changed input, stale proposals, missing evidence, unauthorized changes, insufficient resources and invalid weights. It is not a distributed transaction monitor or a complete adapter for all retained 2.0 interfaces. The machine-readable report identifies which checks actually ran.